جامعة الإمام عبد الرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY

Datra Structure Project Proposal:

Title: SmartMe

Structure: stack

Group : A5

Students' names	Student ID

Instructor:

Dr. Enas EL. Sharawy Dr. Haya Aldossary

Clarification of the Project name and Idea:

The project's name : SmartMe





Arabic and English fonts :

- Droid Arabic kufi | زهراء
- 29Lt azer | زهراء

colors :



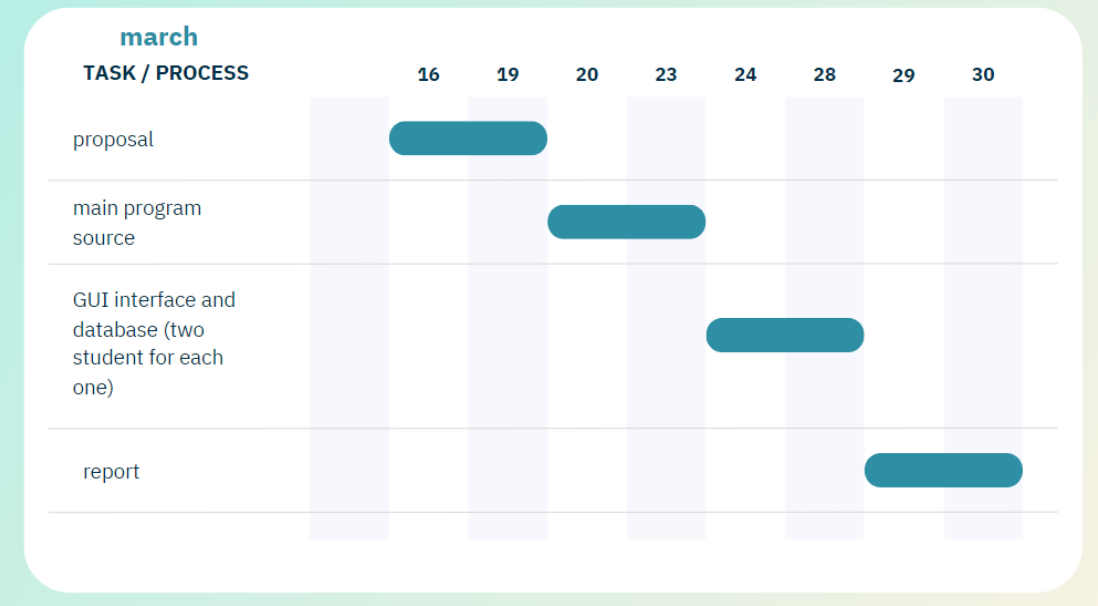
Related words we inspire the logo from : #smartness,#puzzel,#wormth,#hope

Goal of the Program:

-Game Logic Implementation: Develop the game logic to handle card matching and checking for game completion. Use arrays or lists to represent the cards and their positions on the game board.

- **Game Interface Design**: Create a user-friendly interface for the game where players can easily interact with the cards.
- **Score Tracking**: implement a scoring system based on factors like time taken to complete the game and number of moves made.
- **Testing and Debugging**: Test the game thoroughly to identify and fix any bugs or issues that may arise during gameplay.

Gatter table



1. Main Characters and Their Functions:

The player is our main and only charecter , the game has many levels and stages , it gets harder and more complex each level . And if we have to explain the implemented opperations and structures we will find it as followe:

1. Level Initialization:
   - Generating random cards: using simple array data structure.
   - Storing the generated cards: with linked list stack

2. Matching Similar Cards:
   - Insertion: here we will use linked list stack since player should selects a card each time insert so we will need to keep tracking the selected cards.
   - Deletion: If the selected cards match, card need to be removed. and here both array stack and linked list stack can handle it efficiently.

3. Matching Color and Shape:
   - Search: To find cards with a specific color or shape, you'll need to search through the available cards. so Linked list stack will be better to use since it allows easier and faster traversal and searching.

4. Leaderboard:
   - Insertion: When a player finishes a level, you'll need to insert their score and information into the leaderboard. so Linked list stack can be a suitable choice since it allows efficient insertion at the beginning or end of the list.
   - Traversing: To display the leaderboard, we'll need to traverse through the stored scores and information. Linked list stack will be applied here since it provides better traversal capabilities.

5. Timer and Time Expiry:
   - No specific data structure , it can be applied easily using java-

Administrator:

1. Player:

Players can choose from three levels: Beginner, Intermediate, and Advanced. Each level presents its own set of challenges.

player experience :

When players sign in, their information, including their username and password, will be securely stored in a database. After game, the player's highest score achieved will be recorded in the database. A score table will display the top 10 players, providing a competitive element to the game.

Levels:

- Beginner Level:
  In this level, players will match cards based on their symbol and color. For example, they will match a yellow triangle with another yellow triangle.

- Intermediate Level:
  In the intermediate level, players will face more challenging matches. They will need to match cards with different symbols and colors. For example, they might match a yellow triangle with a yellow circle.

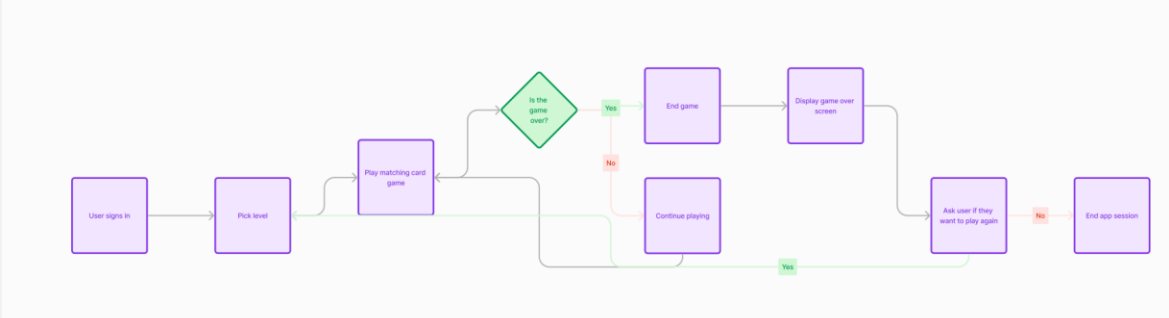
- Advanced Level:
  The advanced level introduces a new gameplay mechanic. Players will select a card from the available options. After selecting a card, all the cards will be displayed again, and the player must identify the card they didn't previously pick.

2. Program Design:

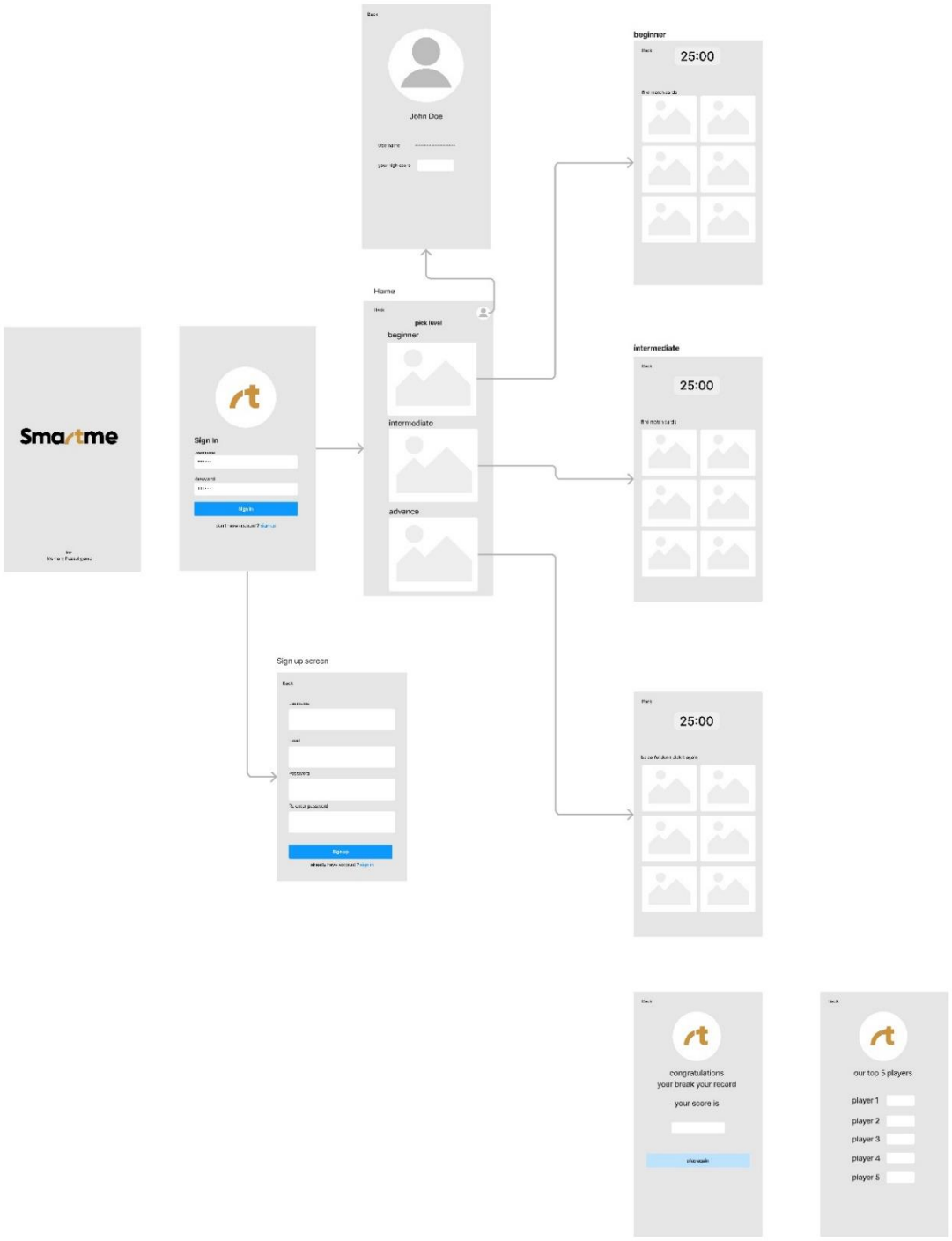
4.1 Architecture:

Stack using array
Stack using linkedlist





4.2 User Interface:



4.4 database schema:

1. Table: Players
    - Columns:
        - player_id (Primary Key): Unique identifier for each player.
        - username: Player's username.
        - password: Encrypted password for login.
        - registration_date: Date and time of player registration.
2. Table: Scores
    - Columns:
        - score_id (Primary Key): Unique identifier for each score.
        - player_id (Foreign Key): References the player_id in the Players table.
        - level: Indicates the level for which the score is recorded (Beginner, Intermediate, Advanced).
        - score: The score achieved by the player.
        - timestamp: Date and time when the score was recorded.
3. Table: Levels
    - Columns:
        - level_id (Primary Key): Unique identifier for each level.
        - level_name: Name of the level (Beginner, Intermediate, Advanced).
        - description: Brief description of the level's challenges.

this schema includes two main tables: Players and Scores. The Players table stores player information, such as username, password, and registration date. The Scores table stores the scores achieved by players, along with the corresponding level, timestamp, and references to the Players table through the player_id.
Additionally, a Levels table is included to store information about each level, such as the level name and a description of its challenges. This table can be useful for displaying level information to players or managing additional level-related data.

3. Conclusion:

In conclusion, the project proposal outlines the development of a memory puzzle game using stack data structures called "SmartMe." Specifically, we will use stack data structures to implement logic, design a user-friendly interface, track scores, and thoroughly test and debug the game. There are several functions described in the proposal, including level initialization, card matching, leaderboard management, and timer functions. Players can also select levels and store scores and information in a database securely in the administrator section. Programs based on stack arrays and stack linked lists are proposed. With the integration of various features and data structures, the project strives to provide an engaging and challenging gaming experience.